

Building GPT-2 from Scratch: Cloze-Style Paraphrase Detection, Sonnet Generation, and a Stylistic Paraphrase Extension (Modern → Shakespearean)

Hasan Semih Selçuk Matin Huseynzade

Department of Computer Engineering, Izmir Institute of Technology (IZTECH)

CENG-534, Default Final Project (adapted from Stanford CS224N)

hasanselcuk@iyte.edu.tr, matinhuseynzade@iyte.edu.tr

Abstract

We implement the GPT-2 architecture from scratch (masked multi-head self-attention, the pre-LayerNorm transformer block, and a decoupled-weight-decay AdamW optimizer) and verify it against the course unit tests. Starting from pre-trained weights, we fine-tune the model on three downstream tasks. For sentiment classification we reach a dev accuracy of **0.967** on CFIMDB (full-model). For paraphrase detection we cast the Quora binary task as cloze-style next-token prediction and obtain **0.889** dev accuracy. For sonnet generation we run a decoding-temperature ablation and reach **41.49** dev chrF at temperature 1.0. As an original extension we build a *stylistic paraphrase generator* that rewrites modern English into Shakespearean English; with source-side loss masking it improves from a zero-shot baseline of 13.85 to **35.83** chrF (+21.97). Beyond raw numbers we contribute two analyses: (i) a prompt byte-pair-encoding “footgun” in which a trailing space in the prompt costs 6.3 chrF, and (ii) a memory recipe for stable variable-length full fine-tuning on the Apple Silicon MPS backend, where we trained without any CUDA GPU.

1 Introduction

Large pretrained language models such as GPT-2 [7] have become the default backbone for both language understanding and generation. The goal of this project is twofold: first, to *understand* GPT-2 by building it from its primitive components rather than calling a library; and second, to *adapt* it, through fine-tuning, to three downstream tasks of increasing generative difficulty (sentiment classification, paraphrase detection, and sonnet generation), and finally to a novel extension of our own.

A recurring theme of the project is the distinction between *detecting* a property of text (classification) and *producing* text with a target property (conditional generation). Paraphrase detection sits on the classification side, while our extension (rewriting modern English into Shakespearean style) sits squarely on the generation side and re-uses the same autoregressive machinery as sonnet generation. This contrast motivated the design of the extension.

Contributions.

1. A complete, unit-test-verified implementation of GPT-2: scaled dot-product masked attention, the pre-LayerNorm transformer block, and AdamW with decoupled weight de-

cay (§3.1).

2. A consistent cloze-prompt formulation for paraphrase detection together with a sequence-length / left-truncation analysis that we found necessary for both accuracy and memory (§3.3).
3. A decoding-temperature ablation for sonnet generation (§3.4, Table 5).
4. **Extension:** stylistic paraphrase generation built on new parallel data with source-side loss masking, reusing the sonnet model with *no new model class* (§3.5).
5. **Two analysis findings:** a prompt whitespace-tokenization footgun worth +6.3 chrF, and a practical memory recipe for full fine-tuning on Apple Silicon MPS (§6).

2 Related Work

GPT-2 [7] is a decoder-only transformer [10] trained with a causal language-modeling objective; we re-implement its forward pass and train it with AdamW [5]. Casting classification as a fill-in-the-blank (cloze) language-modeling problem follows the prompt-as-task line of work, most directly PET [8] and the few-shot framing of GPT-3 [1]; our paraphrase head reduces a binary decision to the model’s preference between the tokens *yes* and *no*. For open-ended generation we use nucleus (top- p) sampling [2] to avoid the degeneration that greedy and low-temperature decoding induce, and evaluate generation with the character- n -gram metric chrF [6], which is well suited to the archaic morphology of our targets. Our extension builds on the parallel “No Fear Shakespeare” corpus of Jhamtani et al. [4]. Where they use a copy-enriched sequence-to-sequence model, we instead fine-tune a single pretrained decoder, trading task-specific architecture for transfer from pre-training. Because we observed instability in multi-epoch full fine-tuning on our hardware, parameter-efficient methods such as LoRA [3] are a natural avenue for future work. The sentiment datasets are SST [9] and CFIMDB.

3 Approach

3.1 GPT-2 implementation

We implement the model bottom-up. **Masked multi-head self-attention** computes, per head,

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)V, \quad (1)$$

where M is a causal mask ($M_{ij} = -\infty$ for $j > i$) that forbids attending to future positions, and the $h=12$ heads are concatenated and projected. Each **transformer block** applies attention and a GELU MLP in the *pre-LayerNorm* configuration with residual connections, $x \leftarrow x + \text{Attn}(\text{LN}(x))$ and $x \leftarrow x + \text{MLP}(\text{LN}(x))$. The **full model** sums token and positional embeddings, stacks $L=12$ blocks, applies a final LayerNorm, and projects to the vocabulary with weights tied to the input embedding.

Our **AdamW** step implements bias-corrected first/second moments with *decoupled* weight decay applied directly to the parameters [5]:

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1-\beta_2^t}, \quad (2)$$

$$\theta_t = \theta_{t-1} - \eta \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}} + \lambda \theta_{t-1} \right). \quad (3)$$

We write the textbook bias-correction form for clarity; our code uses the mathematically equivalent *efficient* update of [5] (folding the correction into the effective step size $\alpha_t = \eta \sqrt{1-\beta_2^t} / (1-\beta_1^t)$ and applying weight decay with the original learning rate), which is exactly what the reference unit test verifies. Both `optimizer_test.py` and `sanity_check.py` pass (“*Your GPT2 implementation is correct!*”), confirming the forward pass and the optimizer numerically match the reference.

3.2 Sentiment classification

We predict a sentiment label from the hidden state of the *last* token of the sequence, passed through dropout and a linear head. We compare two regimes: **last-linear-layer** (the GPT-2 body is frozen and only the head is trained) and **full-model** fine-tuning (all parameters updated). We evaluate on SST-5 (5 classes) [9] and CFIMDB (binary).

3.3 Cloze-style paraphrase detection

We reduce Quora binary paraphrase classification to next-token prediction. A shared template (`make_cloze_prompt`) ends with a position at which the model must emit either `yes` (BPE id 8505) or `no` (id 3919); the predicted label is whichever token receives higher probability.

Design decision: template consistency. Training and evaluation use the *identical* cloze template. An earlier mismatch made dev look strong while the held-out test score dropped. Consistency between train- and test-time prompts proved essential.

Sequence length and left truncation. We cap inputs at `max_length=128` with `truncation_side='left'`. Left truncation is critical: it preserves the *end* of the cloze prompt (where the answer is produced) instead of discarding it. This helped both accuracy and peak memory (§6).

3.4 Sonnet generation

We fine-tune the model for autoregressive language modeling on Shakespeare’s sonnets. At test time the model is conditioned on the first three lines of a sonnet and generates the

remainder with nucleus (top- p) sampling [2] at temperature τ . We evaluate the generated continuations against held-out gold with chrF [6] and ablate τ .

3.5 Extension: stylistic paraphrase generation

Task. Rewrite a modern English sentence into Shakespearean English. Unlike paraphrase *detection*, this is conditional *generation*. We deliberately reuse the SonnetGPT infrastructure (teacher forcing, top- p generation, weight-tied projection), so that **no new model class is introduced**, to isolate the effect of data and training objective.

Data. We use the parallel “No Fear Shakespeare” corpus [4] (harsh19/Shakespeareizing-Modern-English), downloaded directly with `urllib`: **18,395 / 1,218 / 1,462** aligned train/valid/test pairs after a light de-tokenization and an alignment assertion.

Prompt and source-side loss masking. Each example is formatted as Modern: {m}\nShakespearean: {s}<eos>. Crucially, the loss is computed *only over the target (Shakespearean) tokens*: prompt and source tokens are masked with `ignore_index = -100`, with the prompt length computed per example. Concretely:

```
logits = model(x)[: , :-1]
labels = x[: , 1:]
labels[: , :prompt_len-1] = -100 # mask source
labels[padding_positions] = -100 # mask padding
loss = cross_entropy(logits, labels,
                     ignore_index=-100)
```

This focuses the gradient on *producing* the style rather than on reconstructing the (given) source sentence.

4 Experiments

4.1 Data

Sentiment. SST-5 (8,544 train sentences) and CFIMDB (1,707 train documents), using the course-provided splits.

Paraphrase. The Quora paraphrase CSVs shipped in our `data/` directory, containing 283,011 train / 40,430 dev / 80,860 test question pairs.¹ **Sonnet.** 143 training sonnets and a 12-sonnet held-out *test* split (handout §7.2). The starter data additionally ships a held-out *dev* set with gold (`TRUE_sonnets_held_out_dev.txt`), which we use for temperature tuning; the official 12-sonnet test set is held out with hidden labels. **Extension.** Jhamtani parallel corpus, 18,395/1,218/1,462. Metrics: accuracy for classification, chrF for generation.

4.2 Setup and hyperparameters

Hardware. All experiments ran on an **Apple M4 Pro (24 GB unified memory)**, with **no CUDA GPU**, using the PyTorch 2.12.0 MPS backend. Adapting variable-length full

¹The handout (§7.2) quotes a smaller subset (141,506/20,215/40,431); the CSVs distributed with our starter code contain almost exactly twice as many *unique* pairs. We verified this is not a duplication artifact (the dev file has 40,430 distinct ids and 40,430 distinct, order-insensitive sentence pairs) and use the provided files in full; all dev accuracies are measured on this dev file ($n=40,429$ labelled).

Task	Configuration
Sentiment (last-lin.)	head only, batch 8
Sentiment (CFIMDB full)	batch 4, max_len 256, 5 ep.
Paraphrase	batch 16, max_len 128 (L-trunc), 3 ep.
Sonnet	batch 8, 10 ep., top- p 0.9
Extension	batch 16, max_len 128, 5 ep.

Table 1: Per-task training configuration (AdamW, lr 10^{-5} for fine-tuning; “L-trunc” = left truncation).

fine-tuning to MPS was a genuine engineering constraint and is discussed in §6. Table 1 summarizes the per-task configuration; the optimizer is AdamW throughout (lr = 10^{-5} for fine-tuning).

5 Results

All accuracies and chrF scores below are our own measurements on the course dev/held-out splits.

Sentiment (Table 3). Full-model fine-tuning beats the frozen-body head on both datasets, dramatically so on the binary CFIMDB task ($0.857 \rightarrow 0.967$). On the harder 5-class SST the absolute numbers are lower ($0.461 \rightarrow 0.513$), as expected for fine-grained sentiment.

Paraphrase (Table 4). The cloze formulation improves steadily to 0.889 dev accuracy at epoch 2. We stress that this number is measured on our larger dev split (about twice the handout subset; see footnote 1), so it is *not* directly comparable to handout baselines computed on the smaller subset.

Sonnet (Table 5). Sweeping decoding temperature shows chrF rising monotonically to $\tau = 1.0$ (chrF 41.49). The gap to $\tau = 1.2$ (41.12) is only 0.37 chrF, within single-run noise, so we select $\tau = 1.0$ for the test submission rather than claiming a sharp optimum. As a second, non-chrF axis, fine-tuning lowers held-out dev *perplexity* from 99.1 (zero-shot pretrained GPT-2) to 59.0 (lower is better). We report this for reproducibility and to show fine-tuning helps on a second metric, not as a standalone claim of absolute quality.

Extension (Tables 6–7). Fine-tuning lifts chrF from 13.85 to 35.83 (+21.97). Both the zero-shot baseline and the fine-tuned model are evaluated with the *corrected* (no-trailing-space) prompt and identical decoding, so this gain is not confounded by the whitespace effect studied in Table 7. A single prompt-design change accounts for 6.3 of the fine-tuned points.

6 Analysis

Qualitative: stylistic paraphrase. Table 2 shows representative model outputs against gold. The model reliably injects archaic morphology (*hath*, *doth*, *thou*) and re-orders clauses toward an early-modern register, while staying faithful to the source meaning. Failures are dominated by occasional

sampling-induced repetition (e.g. “pray you, pray you”) rather than semantic drift.

The prompt whitespace footgun (+6.3 chrF). Our single most instructive finding (Table 7). GPT-2’s byte-pair encoding attaches a leading space to the *following* token (“Shakespearean” is a different token than “Shakespearean”). When the prompt tag *ended* with a space (“Shakespearean: ”), the model’s first decoding step had to emit a token with no leading space, an out-of-distribution situation, and it frequently produced junk artifacts (~~~~, ---, _____) before recovering. Moving the space off the prompt and onto the target (tag “Shakespearean:”, target “{s}”) aligns generation with the BPE convention and recovers 6.3 chrF. This is a subtle, reproducible tokenization pitfall that generalizes to any prompted GPT-2 generation.

Decoding behaviour. The temperature sweep (Table 5) traces the degeneration/divergence trade-off of [2]: at $\tau = 0.7$ the sonnet model shows visibly more repetition (e.g. “this, this...”), consistent with the lower chrF; at $\tau = 1.2$ it begins to wander; $\tau = 1.0$ balances diversity and fidelity. We quantify this with corpus-level diversity of the dev generations (pooled n -grams over the 12 dev sonnets, fixed seed and conditioning): the fraction of repeated 4-grams falls from 0.17 at $\tau=0.7$ to 0.008 at $\tau=1.0$ and 0.002 at $\tau=1.2$, while distinct-2 rises from 0.51 to 0.81 to 0.88. The low-temperature repetition is therefore real and monotone, not a cherry-picked example; these remain single-run observations. There is also a methodological tension worth stating: chrF is an n -gram-overlap metric, the very family [2] argue underweights degeneration, so we read the temperature trend together with the qualitative samples rather than relying on chrF alone. We use chrF because it is the course’s grading metric.

Systems finding: full fine-tuning on Apple Silicon (MPS). With no CUDA GPU available, we trained entirely on the MPS backend, where variable-length batches fragment the memory allocator: because each batch has a different shape, the cached pool grows until it triggers an out-of-memory error (we observed peaks above 25 GiB on CFIMDB and a crash mid-epoch on the 22k-batch Quora run). Our recipe for stability was: (i) *bound the sequence length* and use small batches; (ii) call `torch.mps.empty_cache()` periodically (every 10–50 steps); and (iii) wrap all evaluation in `torch.no_grad()`. With these in place, CFIMDB full fine-tuning and the 3-epoch paraphrase run completed without OOM. We report this as a practical recipe we found necessary for GPU-free full fine-tuning, not as a novel method.

Does batch size explain the CFIMDB gap? Table 3 noted that CFIMDB full (0.967) trails the handout (0.976), and we hypothesised the small batch (4, forced by MPS memory) was the cause. We tested this with a controlled gradient-accumulation experiment: holding seed, learning rate, epochs (5), dropout, and micro-batch fixed, we raised the *effective* batch from 4 to 32 by accumulating over 8 micro-

Modern (input)	Gold (reference)	Our model (generated)
I love rich Capulet’s daughter.	Then plainly know my heart’s dear love is set / On the fair daughter of rich Capulet.	I love rich Capulet’s daughter.
Holy Saint Francis, this is a drastic change!	Holy Saint Francis, what a change is here!	Saint Francis, this is a sudden change!
Have you given up so quickly on Rosaline, whom you loved so much?	Is Rosaline, whom thou didst love so dear, / So soon forsaken?	Have you no more of Rosalind, whom thou loved so much?

Table 2: Qualitative stylistic-paraphrase examples (modern $\rightarrow$ Shakespearean, §6). The model adopts archaic forms (*thou, didst*) and a period register while preserving meaning.

Dataset	Last-linear		Full-model	
	acc	F1	acc	F1
SST-5	0.461	0.366	0.513	0.467
CFIMDB	0.857	0.857	0.967	0.967

Table 3: Sentiment classification, dev accuracy and macro-F1. The handout dev baselines are 0.462/0.513/0.861/0.976 (SST last / SST full / CFIMDB last / CFIMDB full). We match SST full exactly (0.513) and SST last to within rounding (0.461 vs. 0.462); CFIMDB last is marginally lower (0.857 vs. 0.861). Since we report a single run, this small gap is within expected run-to-run variation and is not directly comparable. The one real gap is CFIMDB full (0.967 vs. 0.976); we initially hypothesised this stemmed from the reduced batch size (4) we needed to fit MPS memory, but a controlled experiment (§6) does *not* support that explanation.

Model	Epoch 0	Epoch 1	Epoch 2
GPT-2 small, full FT, cloze	0.863	0.879	0.889

Table 4: Paraphrase detection (Quora, dev accuracy). Accuracy rises monotonically over three epochs of full fine-tuning with the consistent cloze template and left truncation.

Temperature τ	0.7	0.9	1.0	1.2
dev chrF	39.01	40.63	41.49	41.12

Table 5: Sonnet generation, dev chrF (top- $p = 0.9$ fixed). chrF improves monotonically up to $\tau = 1.0$; the 1.0 vs. 1.2 difference is only 0.37 chrF, within single-run noise, so we select $\tau = 1.0$. Lower temperature (0.7) shows visibly more repetition.

Model	chrF
Zero-shot pretrained GPT-2 (no fine-tune)	13.85
Fine-tuned (source-side masking)	35.83
Δ (gain)	+21.97

Table 6: Extension: stylistic paraphrase (test chrF, $n = 1462$). Fine-tuning with source-side loss masking more than doubles the zero-shot chrF.

batches. The accumulation-step-1 control reproduced our original score exactly (0.9673), confirming the implementation. The per-epoch dev accuracies tell the story: effective batch 4 gives 0.967, 0.967, 0.947, 0.927, 0.910 (it peaks within one epoch and then overfits), whereas effective batch 32 gives

Prompt design	chrF
"Shakespearean: " (trailing space)	29.52
"Shakespearean: " + " {target}"	35.83

Table 7: Extension ablation: prompt design (test chrF). Removing the trailing space from the prompt tag, and instead prefixing the space to the target, is worth +6.3 chrF (see §6).

0.551, 0.653, 0.922, 0.947, 0.959 (it climbs monotonically and is *still improving* at the 5-epoch budget, not yet converged). So a larger batch does *not* reach a higher score within the same budget; the data therefore *do not support* the hypothesis that the small batch caused the gap. We are careful not to over-read this: the two final scores differ by only 0.008 on a single seed and the large-batch run had not converged, so we cannot claim the larger batch is genuinely *worse*, only that it fails to close the gap. The residual gap to 0.976 is unexplained and, under our single-seed protocol, may reflect initialisation or other run-to-run variation. We consider this informative: a plausible hypothesis, tested rather than asserted.

Limitations. We use the small GPT-2 (124M). The sonnet corpus is tiny (143 sonnets), so the model overfits beyond ~ 2 epochs (dev chrF degrades). Multi-epoch full fine-tuning was unstable on MPS for the largest runs, which is precisely the regime where parameter-efficient methods such as LoRA [3] would help, both for memory and for overfitting.

7 Conclusion and Future Work

We built GPT-2 from its primitives and fine-tuned it across sentiment classification (CFIMDB dev acc 0.967), cloze-style paraphrase detection (Quora dev acc 0.889), and sonnet generation (dev chrF 41.49), then extended it to a stylistic paraphrase generator that improves from 13.85 to 35.83 chrF (+21.97). Two analyses, a 6.3-chrF prompt tokenization foot-gun and a memory recipe for MPS full fine-tuning, turned out to matter as much as the headline numbers. Future work: parameter-efficient fine-tuning (LoRA [3]) to enable a larger backbone (GPT-2 medium) and more stable multi-epoch training; preference-based alignment (e.g. DPO) for sonnet quality; and a wider top- p / temperature search.

A Reproducing the Results

Every number in Tables 3–7 was computed by us on the dev / held-out splits and can be regenerated as follows. All com-

mands run from the repository root with the course environment active (conda activate cs224n_dfp; export PYTORCH_ENABLE_MPS_FALLBACK=1). The full procedure is also in REPRODUCE.md.

Fast path (seconds, CPU, no retraining). Re-score the saved predictions in predictions/ against the gold labels in data/; this reproduces Tables 3–6 exactly:

```
python reproduce_metrics.py
```

The script loads no model: it applies scikit-learn accuracy/F1 and the course evaluation.test_sonnet / sacrebleu CHRF scorers to the committed artifacts, so the reported numbers are independently checkable.

Full re-training / generation. The runs that produced those artifacts:

```
# Part 0: implementation sanity checks
python optimizer_test.py && python sanity_check.py

# Table 3: sentiment (dev acc / macro-F1)
python classifier.py \
  --fine-tune-mode last-linear-layer --use_gpu --lr 1e-3
python classifier.py \
  --fine-tune-mode full-model --use_gpu --lr 1e-5

# Table 4: paraphrase (Quora dev acc)
python paraphrase_detection.py \
  --use_gpu --epochs 3 --batch_size 16 --lr 1e-5

# Table 5: sonnet (dev chrF temperature sweep)
python sonnet_generation.py --use_gpu --epochs 10 --lr 1e-5
for T in 0.7 0.9 1.0 1.2; do
  python eval_sonnet_dev.py --use_gpu \
    --temperature $T --top_p 0.9
done

# Tables 6-7: stylistic paraphrase extension
python prepare_style_data.py
python stylistic_paraphrase.py \
  --use_gpu --epochs 5 --batch_size 16 --lr 1e-5

# Sonnet diagnostics: held-out perplexity (99.1
# zero-shot -> 59.0 FT) + per-temperature
# corpus-level distinct-n / rep-4 (no retraining;
# uses the saved checkpoint)
python analyze_sonnet.py --use_gpu

# CFIMDB batch-size hypothesis test (Analysis):
# effective batch 4 vs 32 via
# gradient accumulation, all else fixed -> 0.9673
# vs 0.9592 (hypothesis not supported)
python run_cfimdb_batch.py all
```

Test-set scores are intentionally omitted: the official test labels are hidden, so we report dev / held-out numbers throughout.

References

- [1] Tom B Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [2] Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. The curious case of neural text degeneration. In *International Conference on Learning Representations (ICLR)*, 2020.
- [3] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.
- [4] Harsh Jhamtani, Varun Gangal, Eduard Hovy, and Eric Nyerberg. Shakespearizing modern language using copy-enriched sequence-to-sequence models. In *Proceedings of the Workshop on Stylistic Variation*, pages 10–19, 2017.
- [5] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019.
- [6] Maja Popović. chrF: character n-gram f-score for automatic mt evaluation. In *Proceedings of the Tenth Workshop on Statistical Machine Translation*, pages 392–395, 2015.
- [7] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. *OpenAI Technical Report*, 2019.
- [8] Timo Schick and Hinrich Schütze. Exploiting cloze-questions for few-shot text classification and natural language inference. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics (EACL)*, pages 255–269, 2021.
- [9] Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D Manning, Andrew Ng, and Christopher Potts. Recursive deep models for semantic compositionality over a sentiment treebank. In *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1631–1642, 2013.
- [10] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.